

AGENT IDENTITY FOR AI SYSTEMS

Microsoft Entra Agent ID vs AWS AgentCore Identity
A Complete Technical Research with Code & Comparison

Microsoft Entra Agent ID

AWS AgentCore Identity

Enterprise Cloud Architecture Research • June 2026 • v1.0

Executive Summary

As AI agents move from prototypes to enterprise production, they expose a fundamental gap: existing identity systems designed for humans or traditional applications are insufficient for the dynamic, autonomous, and often ephemeral nature of AI workloads. In 2025–2026, both Microsoft and Amazon Web Services released dedicated agent identity platforms — **Microsoft Entra Agent ID** (GA: May 2026) and **AWS AgentCore Identity** (GA: October 2025) — both reaching enterprise maturity within months of each other.

This research provides a deep technical comparison: architecture, authentication flows, governance models, code patterns, and a decision framework for architects choosing between, or combining, both systems in multi-cloud deployments.

Key Finding: The two platforms are designed for interoperability. AWS AgentCore Identity and Microsoft Entra Agent ID can be federated via OIDC/WIF, enabling AWS-native agents to carry Entra-governed identities — the recommended pattern for multi-cloud enterprise deployments.

1. What is Agent Identity?

Traditional identity systems recognize two principal types: **human users** (with interactive login, MFA, and session-based tokens) and **service accounts / app registrations** (with client secrets or certificates). AI agents fit neither model cleanly:

- Agents may be **ephemeral** — created and destroyed thousands of times per day
- Agents operate **autonomously** without interactive user sessions
- Agents may **act on behalf of users** without users being present
- Agents must call **multiple downstream services** with scoped, least-privilege tokens
- Agents need **auditability** — who the agent acted as, when, and what it accessed

Agent Identity systems solve this by introducing purpose-built identity constructs that support dynamic lifecycle management, delegated user authority (OBO), credential-less authentication via federation, and enterprise governance — all applied to non-human principals.

2. Microsoft Entra Agent ID — Deep Dive

Microsoft Entra Agent ID reached general availability on May 1, 2026 as part of the broader Microsoft Entra suite. It extends Entra's existing identity infrastructure — the same system that authenticates over 1.2 billion sign-ins daily — to AI agents, introducing three new identity constructs and specialized OAuth flows.

2.1 Core Constructs

Entra Agent ID introduces a three-tier identity hierarchy:

Agent Identity Blueprint:

A reusable template defining the security envelope for a class of agents. Sets allowed scopes, lifecycle policies (max lifetime), owner/sponsor assignments, and Conditional Access requirements. Think of it as an IAM policy template for agents.

Agent Identity:

The actual identity instance created from a Blueprint. A special service principal in Entra ID with no credentials of its own — tokens are acquired via the Blueprint. Can exist for minutes or months depending on the lifecycle policy.

Agent User Account:

A backing user identity for on-behalf-of (OBO) scenarios. Enables an agent to acquire user-scoped tokens that carry the user's identity claims to downstream services.

■ *Creating an Agent Identity Blueprint via Microsoft Graph API (Python)*

```
import requests

# Step 1: Authenticate with Microsoft Entra (client credentials)
TOKEN_URL = "https://login.microsoftonline.com/{tenant_id}/oauth2/v2.0/token"
creds = {
    "client_id": "<client_id>",
    "client_secret": "<client_secret>",
    "scope": "https://graph.microsoft.com/.default",
    "grant_type": "client_credentials"
}
token_resp = requests.post(TOKEN_URL, data=creds)
access_token = token_resp.json()["access_token"]

headers = {
    "Authorization": f"Bearer {access_token}",
    "Content-Type": "application/json"
}

# Step 2: Create Agent Identity Blueprint
blueprint_payload = {
    "displayName": "FinanceAnalystAgent",
    "description": "Blueprint for financial analysis agents",
    "accessEnvelope": {
        "allowedScopes": ["User.Read", "Files.Read", "Mail.Read"]
    },
    "lifecyclePolicy": {
        "maxLifetimeDays": 90
    }
}

resp = requests.post(
    "https://graph.microsoft.com/v1.0/agentIdentityBlueprints",
    headers=headers,
    json=blueprint_payload
)
blueprint_id = resp.json()["id"]
print(f"Blueprint created: {blueprint_id}")

# Step 3: Instantiate an Agent Identity from the Blueprint
agent_payload = {"blueprintId": blueprint_id, "displayName": "FinAgent-001"}
agent_resp = requests.post(
    "https://graph.microsoft.com/v1.0/agentIdentities",
    headers=headers,
    json=agent_payload
)
```

```
print(f"Agent Identity ID: {agent_resp.json()['id']}")
```

2.2 Workload Identity Federation with AWS

For AWS-hosted agents that need Microsoft resource access, Entra Agent ID supports **Workload Identity Federation (WIF)** — allowing an agent to exchange an AWS STS OIDC token directly for an Entra access token, with no secrets stored anywhere.

■ AWS Agent exchanging STS OIDC token for Microsoft Entra token (WIF pattern)

```
import boto3
import requests

# Step 1: AWS agent acquires its own STS OIDC token
sts_client = boto3.client("sts", region_name="us-east-1")
# Note: assumes execution role with sts:GetFederationToken permission
assumed = sts_client.assume_role_with_web_identity(
    RoleArn="arn:aws:iam::123456789012:role/AgentEntraFederationRole",
    RoleSessionName="agent-session",
    WebIdentityToken=open("/var/run/secrets/eks.amazonaws.com/serviceaccount/token").read()
)
aws_oidc_token = assumed["Credentials"]["SessionToken"]

# Step 2: Exchange AWS OIDC token for Microsoft Entra token via WIF
TENANT_ID = "your-tenant-id"
ENTRA_APP_ID = "your-entra-app-id"

entra_resp = requests.post(
    f"https://login.microsoftonline.com/{TENANT_ID}/oauth2/v2.0/token",
    data={
        "client_id": ENTRA_APP_ID,
        "grant_type": "urn:ietf:params:oauth:grant-type:jwt-bearer",
        "client_assertion_type": "urn:ietf:params:oauth:client-assertion-type:jwt-bearer",
        "client_assertion": aws_oidc_token, # AWS token as the credential
        "scope": "https://graph.microsoft.com/.default",
        "requested_token_use": "on_behalf_of"
    }
)
entra_token = entra_resp.json()["access_token"]
print("Entra token acquired – no secrets stored anywhere")

# Step 3: Call Microsoft Graph as the agent
graph_resp = requests.get(
    "https://graph.microsoft.com/v1.0/me/messages?$top=5",
    headers={"Authorization": f"Bearer {entra_token}"}
)
emails = graph_resp.json()["value"]
print(f"Retrieved {len(emails)} emails as agent identity")
```

2.3 Conditional Access & Governance

A key differentiator of Entra Agent ID is that agents receive the same Conditional Access policies as human users — real-time risk signals, location controls, and device compliance requirements can all be applied to agent token issuance. Lifecycle Workflows automate sponsor notifications and access expiry. All agent authentication events appear in Entra Sign-in Logs and Microsoft Sentinel.

3. AWS AgentCore Identity — Deep Dive

AWS AgentCore Identity reached general availability in October 2025, several months ahead of Microsoft's offering. It is a standalone service within Amazon Bedrock AgentCore that provides identity and credential management for agents running anywhere — on AWS compute (ECS, Lambda, EKS), on-premises, or on other clouds.

3.1 Core Constructs

AgentCore Identity is built around **Workload Identities** — a stable identity anchor that abstracts multiple credential types (IAM, OAuth2, API keys) behind a unified interface. Key components:

Workload Identity:

The primary identity object for an agent. Created automatically when deploying via AgentCore Runtime/Gateway, or manually. Identified by a Workload Identity ARN.

Token Vault:

Secure storage for OAuth refresh tokens, enabling agents to maintain long-lived access to third-party services without re-authorization.

Credential Providers:

Configured connections to external OAuth services (Microsoft Graph, Salesforce, etc.). Agents call `GetResourceOAuth2Token` to receive a scoped token for each downstream service.

Secrets Manager Integration:

As of June 2026, customers can reference existing Secrets Manager ARNs in Credential Providers with CMK encryption and automatic rotation.

■ *Creating a Workload Identity and getting an access token (Python + boto3)*

```
import boto3

# Initialize AgentCore Identity client
agentcore = boto3.client("bedrock-agentcore", region_name="us-east-1")

# Create a workload identity for your agent
response = agentcore.create_workload_identity(
    name="finance-analyst-agent",
    description="Identity for the Q2 financial analysis agent",
    allowedClientIds=["arn:aws:iam::123456789012:role/AgentExecutionRole"],
    tags={"Environment": "production", "Team": "finance", "CostCenter": "AI-Ops"}
)
```

```

workload_identity_arn = response["workloadIdentityArn"]
print(f"Created: {workload_identity_arn}")

# Get a workload access token (M2M – agent acting as itself)
token_resp = agentcore.get_workload_access_token(
    workloadIdentityArn=workload_identity_arn
)
access_token = token_resp["accessToken"]
print(f"Token expires at: {token_resp['expiresAt']}")

# Configure a credential provider for Microsoft Graph access
cred_provider = agentcore.create_credential_provider(
    name="msgraph-provider",
    credentialProviderType="OAUTH2",
    oauth2Config={
        "authorizationEndpoint": "https://login.microsoftonline.com/{tenant}/oauth2/v2.0/authorize",
        "tokenEndpoint": "https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token",
        "clientId": "<entra_app_client_id>",
        "secretArn": "arn:aws:secretsmanager:us-east-1:123:secret:entra-client-secret",
        "scopes": ["https://graph.microsoft.com/Mail.Read", "https://graph.microsoft.com/Files.Read"]
    }
)
print(f"Credential provider ARN: {cred_provider['credentialProviderArn']}")

```

3.2 On-Behalf-Of (OBO) Token Exchange

Released in April 2026, OBO Token Exchange is the most powerful AgentCore Identity feature: an agent can accept a user's JWT and exchange it for a downstream-scoped token that carries *both* the user's identity and the agent's identity — with no browser redirect and no additional consent prompt.

■ AWS AgentCore OBO Token Exchange — agent acting as the authenticated user

```

import boto3, requests

agentcore = boto3.client("bedrock-agentcore", region_name="us-east-1")
WORKLOAD_ARN =
"arn:aws:bedrock-agentcore:us-east-1:123456789:workloadIdentity/agent-001"

# Inbound: user JWT from your app's auth layer (Cognito, Entra, Okta, etc.)
user_jwt = "<inbound_user_access_token>"

# Step 1: Exchange user JWT for an AgentCore workload access token (OBO)
obo_resp = agentcore.get_workload_access_token_for_jwt(
    workloadIdentityArn=WORKLOAD_ARN,
    jwt=user_jwt
)
agent_token = obo_resp["accessToken"]

```



```
# Step 2: Use AgentCore to get a downstream OAuth token preserving user identity
cred_resp = agentcore.get_resource_oauth2_token(
    workloadIdentityArn=WORKLOAD_ARN,
    credentialProviderName="msgraph-provider",
    oauth2Flow="ON_BEHALF_OF_TOKEN_EXCHANGE", # Key: OBO flow
    scopes=["https://graph.microsoft.com/Files.Read"]
)
downstream_token = cred_resp["accessToken"]
print(f"OBO token obtained – user identity preserved")
print(f"Token subject (user): {cred_resp.get('sub', 'preserved in claims')}")

# Step 3: Call Microsoft Graph AS THE USER via the agent
graph_resp = requests.get(
    "https://graph.microsoft.com/v1.0/me/drive/root/children",
    headers={"Authorization": f"Bearer {downstream_token}"})
files = graph_resp.json().get("value", [])
print(f"Listed {len(files)} files from user's OneDrive")
# SharePoint/Graph sees the USER's identity, not just the bot's
```

3.3 Bring Your Own Secrets (June 2026)

AgentCore Identity now supports referencing existing Secrets Manager ARNs directly, enabling customers to apply organization-specific governance: custom CMKs, tagging strategies, automatic rotation policies, and resource-level IAM — without changing how AgentCore uses them at runtime.

■ *Configuring a Credential Provider with customer-managed secret (BYO Secrets)*

```
import boto3

secrets_mgr = boto3.client("secretsmanager", region_name="us-east-1")
agentcore = boto3.client("bedrock-agentcore", region_name="us-east-1")

# Step 1: Create your own secret with CMK and rotation policy
secret = secrets_mgr.create_secret(
    Name="agentcore/prod/entra-client-secret",
    SecretString='{"client_secret": "your-secret-value"}',
    KmsKeyId="arn:aws:kms:us-east-1:123:key/cmkey-id", # Customer-managed key
    Tags=[
        {"Key": "Environment", "Value": "production"},
        {"Key": "ManagedBy", "Value": "AgentCore"},
        {"Key": "CostCenter", "Value": "AI-Ops"}
    ]
)

# Enable automatic rotation
secrets_mgr.rotate_secret(
    SecretId=secret["ARN"],
    RotationRules={"AutomaticallyAfterDays": 30}
)
```

```
# Step 2: Reference the existing secret ARN in AgentCore (BYO pattern)
agentcore.create_credential_provider(
    name="entra-provider-cmk",
    credentialProviderType="OAUTH2",
    oauth2Config={
        "tokenEndpoint": "https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token",
        "clientId": "<entra_app_id>",
        "secretArn": secret["ARN"], # Reference existing secret – not service-managed
        "scopes": ["https://graph.microsoft.com/.default"]
    }
)
print("Credential provider configured with BYO CMK-encrypted secret")
```

4. Cross-Cloud Integration Architecture

The most powerful enterprise pattern combines both platforms: agents built and deployed on AWS Bedrock (using Claude models) with a Microsoft Entra Agent ID as their governed identity for accessing Microsoft 365 resources. Both vendors officially support and document this federation.

4.1 Architecture Flow

- 1. User authenticates:** User signs in via your app using Entra ID (or Cognito/Okta). App receives a user JWT.
- 2. Agent invoked:** App calls the AWS-hosted Bedrock agent, passing the user JWT as a bearer token.
- 3. AgentCore validates inbound JWT:** AgentCore Identity validates the inbound token via OIDC against your IdP (Entra/Cognito).
- 4. OBO token exchange:** AgentCore exchanges the user JWT for a workload access token that carries the user's identity.
- 5. WIF federation:** AgentCore calls Entra's token endpoint with the AWS workload token as a federated credential assertion.
- 6. Entra issues token:** Entra Agent ID validates the WIF trust relationship and issues a scoped Microsoft Graph token.
- 7. Agent calls Microsoft APIs:** Bedrock agent calls Graph/SharePoint/Teams with the Entra-issued token. Graph sees the user's identity.
- 8. Full audit trail:** Both AgentCore (CloudWatch) and Entra (Sign-in Logs) record the complete auth chain.

■ Complete cross-cloud agent: AWS Bedrock (Claude) + Microsoft Entra Agent ID identity

```
import boto3, requests, json

class CrossCloudAgent:
    """AWS Bedrock agent governed by Microsoft Entra Agent ID."""

    def __init__(self, tenant_id, entra_app_id, workload_identity_arn):
        self.tenant_id = tenant_id
        self.entra_app_id = entra_app_id
        self.workload_arn = workload_identity_arn
        self.agentcore = boto3.client("bedrock-agentcore", region_name="us-east-1")
        self.bedrock = boto3.client("bedrock-runtime", region_name="us-east-1")

    def get_entra_token(self, user_jwt=None):
        """Get Microsoft Entra token via WIF – no stored secrets."""
```

```

# Get AgentCore workload access token
if user_jwt:
    aws_token = self.agentcore.get_workload_access_token_for_jwt(
        workloadIdentityArn=self.workload_arn, jwt=user_jwt
    )["accessToken"]
else:
    aws_token = self.agentcore.get_workload_access_token(
        workloadIdentityArn=self.workload_arn
    )["accessToken"]

# Exchange for Entra token via Workload Identity Federation
resp = requests.post(
    f"https://login.microsoftonline.com/{self.tenant_id}/oauth2/v2.0/token",
    data={
        "client_id": self.entra_app_id,
        "grant_type": "urn:ietf:params:oauth:grant-type:jwt-bearer",
        "client_assertion_type":
            "urn:ietf:params:oauth:client-assertion-type:jwt-bearer",
        "client_assertion": aws_token,
        "scope": "https://graph.microsoft.com/.default",
        "requested_token_use": "on_behalf_of"
    }
)
return resp.json()["access_token"]

def call_graph(self, endpoint, user_jwt=None):
    """Call Microsoft Graph with federated Entra identity."""
    token = self.get_entra_token(user_jwt)
    return requests.get(
        f"https://graph.microsoft.com/v1.0/{endpoint}",
        headers={"Authorization": f"Bearer {token}"},
    ).json()

def run(self, user_query, user_jwt=None):
    """Invoke Claude on Bedrock with M365 context."""
    # Get user context from Microsoft Graph (as the user via OBO)
    emails = self.call_graph("me/messages?$top=5&$select=subject,from,receivedDateTime", user_jwt)
    files = self.call_graph("me/drive/recent?$top=5&$select=name,lastModifiedDateTime", user_jwt)

    context = {
        "recent_emails": emails.get("value", []),
        "recent_files": files.get("value", [])
    }

    # Invoke Claude on Bedrock with user context
    response = self.bedrock.invoke_model(
        modelId="us.anthropic.claude-3-haiku-20240307-v1:0",
        body=json.dumps({
            "anthropic_version": "bedrock-2023-05-31",

```

```
"max_tokens": 1024,
"system": "You are an enterprise assistant with access to the user's M365 context.",
"messages": [{
  "role": "user",
  "content": f"User M365 context:\n{json.dumps(context, indent=2)}\n\nQuery: {user_query}"
}]
})
)
result = json.loads(response["body"].read())
return result["content"][0]["text"]

# Usage
agent = CrossCloudAgent(
  tenant_id="your-azure-tenant-id",
  entra_app_id="your-entra-app-registration-id",
  workload_identity_arn="arn:aws:bedrock-agentcore:us-east-1:123:workloadIdentity/my-agent"
)
answer = agent.run("Summarize my recent emails about Q2 budget",
  user_jwt="<user_entra_jwt>")
print(answer)
```

5. Feature Comparison

Side-by-side comparison across 12 enterprise dimensions:

Feature	Microsoft Entra Agent ID	AWS AgentCore Identity
GA Date	May 1, 2026	October 2025
Identity Model	Service Principal (Blueprint + Instance)	Workload Identity (auto or manual)
User Delegation	Native OBO via Entra grant	OBO Token Exchange (April 2026)
Secret Mgmt	Keyless/WIF — no stored secrets	Secrets Manager BYO (June 2026, CMK support)
Cross-cloud	Native AWS Bedrock & n8n WIF	Entra ID, Cognito, Okta integration
Audit/Logging	Entra Sign-in Logs + Conditional Access	CloudWatch + OTEL (Datadog, LangSmith)
Governance	Lifecycle Workflows, Entitlement Management	IAM Policies, VPC, Resource Tagging
Auth Protocols	OAuth 2.0, OIDC, SAML	OAuth 2.0, SigV4, OIDC, API Keys
Agent Lifecycle	Blueprint-driven + sponsor oversight	Auto by Runtime/Gateway or manual creation
Pricing	Entra P1/P2 + Agent 365 per-user license	Consumption-based per token operation
OSS Frameworks	Copilot Studio, via SDK/WIF	CrewAI, LangGraph, LlamaIndex, OpenAI SDK
Zero Trust	Conditional Access + real-time risk signals	Per-request verification, least-privilege OBO

6. Decision Framework

Choose Microsoft Entra Agent ID when...

- ✓ Your agents primarily call Microsoft 365, Azure, Teams, or SharePoint APIs
- ✓ Your organization uses Conditional Access policies and wants agents under the same governance
- ✓ You need Blueprint-based identity templates to govern hundreds of agent instances consistently

- ✓ Your identity team manages everything in Entra/Active Directory already
- ✓ You require fine-grained lifecycle workflows with human sponsors for each agent

Choose AWS AgentCore Identity when...

- ✓ Your agents run on AWS compute (ECS, Lambda, EKS) and primarily access AWS services
- ✓ You want consumption-based pricing with no per-seat licensing overhead
- ✓ Your agent framework is open-source (LangGraph, CrewAI, LlamaIndex)
- ✓ You need multi-IdP support from day one (Cognito, Entra, Okta in parallel)
- ✓ You want a bring-your-own-secrets model with CMK and automatic rotation

Use Both (Cross-Cloud Federation) when...

- ✓ Agents live on AWS but need access to Microsoft Graph, SharePoint, or Teams
- ✓ You have a Microsoft-first enterprise (Entra, M365) with AWS-native AI workloads
- ✓ You want zero stored secrets anywhere — use AgentCore WIF + Entra WIF end-to-end
- ✓ Your audit requirements span both AWS CloudTrail/CloudWatch and Microsoft Sentinel
- ✓ You need OBO flows: agent acting as the user across both cloud ecosystems

7. Security Considerations

■ Token Scope Creep

Agent tokens must be scoped to minimum required permissions. Both platforms support fine-grained scope definitions, but require explicit configuration. Default to narrowest scope; expand only when a specific tool requires it.

■ Credential Rotation

AWS AgentCore supports automatic secret rotation via Secrets Manager (30-day cycles). Microsoft Entra Agent ID avoids long-lived credentials entirely via WIF — the preferred approach when available, as it removes rotation risk completely.

■ Token Lifetime

Agent access tokens typically expire in 1 hour (Entra) to 60 minutes (AgentCore). Do not cache tokens beyond their expiry. Use refresh tokens in the Token Vault (AgentCore) or re-acquire via WIF flow (Entra) for long-running agents.

■ Audit & Forensics

Entra logs appear in Microsoft Sentinel and can trigger alerts on anomalous agent behavior. AgentCore logs ship to CloudWatch and are OTEL-compatible for Datadog, LangSmith, and Langfuse. In cross-cloud setups, maintain a unified SIEM view that correlates events from both platforms.

■ Supply Chain Risk (Shadow AI)

Both platforms support shadow AI detection — agents operating outside defined blueprints (Entra) or without registered workload identities (AgentCore) can be detected and blocked. Enforce agent identity registration before any production deployment.

8. Conclusion & Recommendations

Both Microsoft Entra Agent ID and AWS AgentCore Identity have reached enterprise maturity in 2026. They solve the same core problem — giving AI agents first-class, governed, auditable identities — but with different approaches reflecting each vendor's ecosystem strengths.

Critically, they are designed for **interoperability, not competition**. Microsoft explicitly lists AWS Bedrock as a supported third-party platform, and AWS lists Microsoft Entra ID as a

supported identity provider. The recommended enterprise pattern for multi-cloud AI is to use AgentCore Identity as the AWS-side anchor, federated into Entra Agent ID for Microsoft resource access — achieving zero stored secrets, full audit coverage across both clouds, and user-identity preservation via OBO token exchange.

Recommendation: Start with whichever platform your primary workloads run on. If your agents are AWS-native, start with AgentCore Identity and add Entra WIF federation when Microsoft API access is needed. If you are Microsoft-first, use Entra Agent ID Blueprints for governance and add AWS STS federation for Bedrock workloads. The federation trust between both systems can be established in under 30 minutes using published reference implementations from both Microsoft and Amazon.